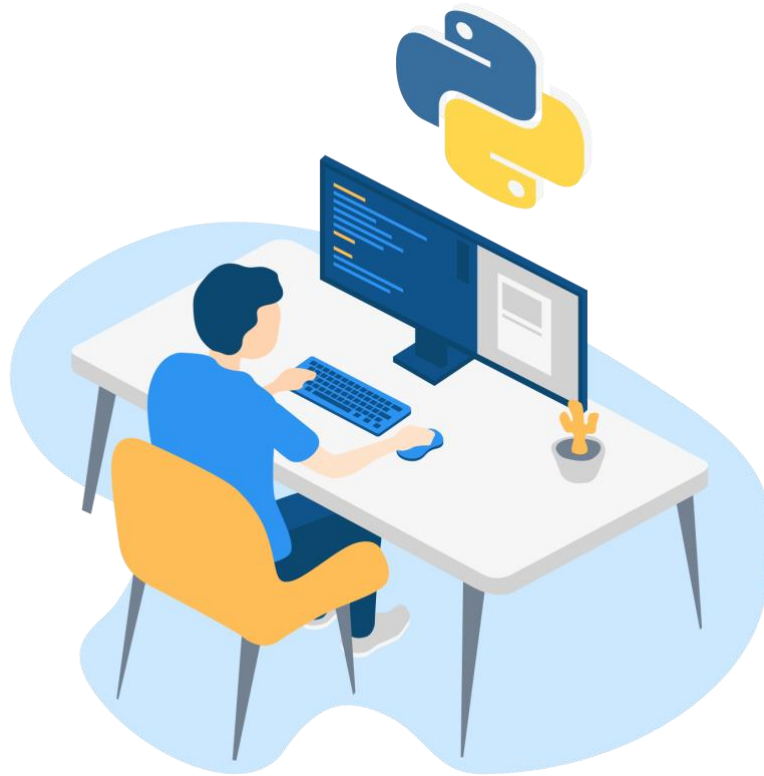




## **10.1 Tujuan**

Setelah menyelesaikan bab ini dan mengikuti kegiatan praktikum, mahasiswa diharapkan mampu:

1. Mengkonfigurasi *media handling* di Django dan mengimplementasikan fungsionalitas *upload* gambar (`ImageField`) pada model `Post` untuk menyertakan gambar pada postingan.
2. Mengintegrasikan *rich text editor* pihak ketiga (`django-tinymce`) untuk menggantikan `TextField` standar
3. Melakukan *refactoring frontend* aplikasi blog menggunakan *framework* Bootstrap

## 10.2 Pengantar

Pada bab-bab sebelumnya, kita telah berhasil membangun aplikasi blog yang fungsional. Pengguna dapat membuat, mengedit, menghapus, dan mengomentari postingan, serta dilindungi oleh sistem autentikasi. Namun, aplikasi kita masih memiliki dua keterbatasan utama: fungsionalitas *input* yang terlalu sederhana (teks polos) dan tampilan yang sangat dasar.

### 10.2.1 Menangani File Media

Hampir semua aplikasi web modern perlu menangani file yang diunggah pengguna, seperti gambar profil, dokumen, atau foto untuk postingan blog. Django sudah menyediakan sistem yang jelas untuk mengelola file-file tersebut. File sendiri terbagi menjadi dua :

- File Statis (**STATIC**):

File-file yang dibutuhkan oleh aplikasi, seperti CSS, JavaScript, dan gambar logo. File-file ini disediakan sebagai pengembang.

- File Media (**MEDIA**): File-file yang diunggah oleh *pengguna*. File-file ini tidak dapat diprediksi dan harus disimpan secara terpisah. Diatur oleh `MEDIA_URL` dan `MEDIA_ROOT`.

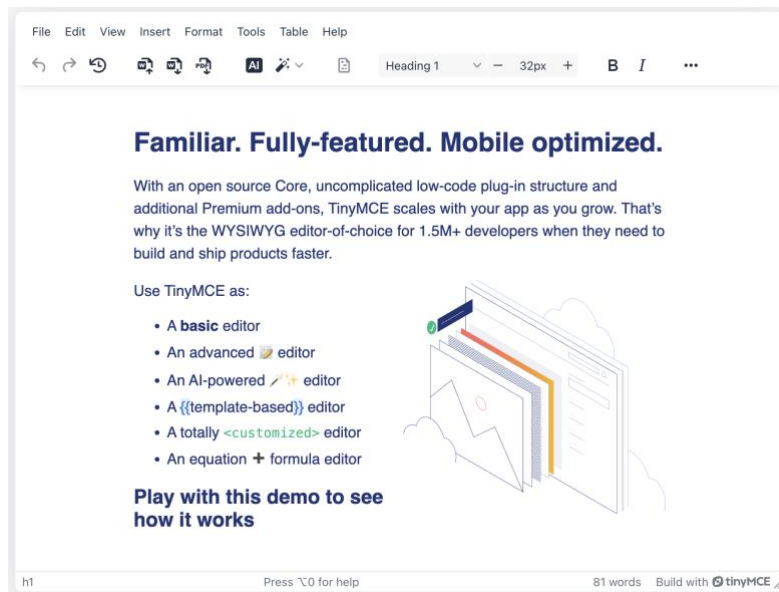
Konfigurasi yang Diperlukan:

1. **Pillow**: Django memerlukan *library* `Pillow` untuk memproses operasi gambar. Library ini harus diinstal melalui `pip`.
2. `MEDIA_ROOT`: Path absolut di sistem ke tempat file yang diunggah pengguna akan disimpan.
3. `MEDIA_URL`: URL publik yang akan digunakan untuk mengakses file-file media tersebut di peramban.
4. `ImageField`: Tipe *field* model khusus yang memvalidasi bahwa file yang diunggah adalah gambar.
5. `enctype="multipart/form-data"`: Atribut ini **wajib** ditambahkan pada tag `<form>` HTML. Tanpa ini, peramban tidak akan mengirimkan data file, dan proses *upload* akan gagal.

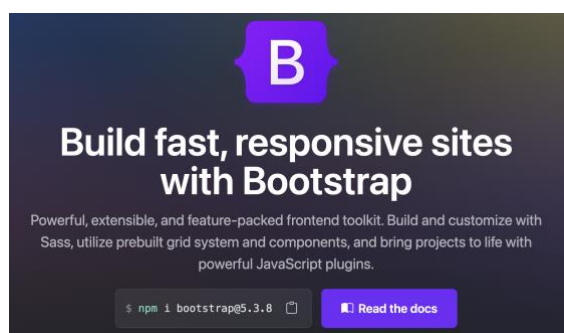
### 10.2.2 Rich Text Editor

Secara *default*, `TextField` di Django hanya menyediakan kotak teks polos. Kotak teks polos ini tidak ideal untuk postingan blog yang mungkin memerlukan *heading*, *bullet points*, teks tebal, atau tautan.

Rich Text Editor (RTE) atau editor WYSIWYG (*What You See Is What You Get*), mengatasi ini dengan menyediakan antarmuka editor yang mirip Microsoft Word di peramban. Editor ini akan menghasilkan HTML, yang kemudian kita simpan di basis data.



### 10.2.3 Bootstrap



Bootstrap adalah sebuah framework (kerangka kerja) CSS gratis dan *open-source* yang ditujukan untuk pengembangan *frontend* (antarmuka pengguna) situs web.

- Framework (Kerangka Kerja): Sebuah *framework* adalah sebuah sistem yang menyediakan struktur dan aturan main, di mana kita dapat memasukkan kode ke dalamnya. Bootstrap menyediakan "cetakan" lengkap untuk tata letak, komponen, dan fungsionalitas.

- CSS: Inti dari Bootstrap adalah sebuah file CSS yang sangat besar dan ditulis dengan sangat baik.
- Frontend: Bootstrap berfokus pada apa yang dilihat dan diinteraksikan oleh pengguna di peramban mereka (HTML, CSS, dan JavaScript).

Analogi sederhananya: Jika HTML adalah kerangka bangunan dan CSS murni adalah cat dan bahan mentah (semen, batu bata), maka Bootstrap adalah komponen modular pra-pabrikasi (dinding siap pasang, kusen jendela standar, unit dapur) yang memungkinkan kita membangun rumah yang fungsional dan terlihat profesional dalam waktu yang jauh lebih singkat.

Tujuan utama Bootstrap adalah untuk kecepatan pengembangan, desain yang responsif, dan konsistensi visual. Menggunakan *framework* seperti Bootstrap menawarkan beberapa keuntungan besar:

1. Desain Responsif (Mobile-First): Bootstrap dirancang dengan filosofi *mobile-first*, artinya *default styling*-nya dioptimalkan untuk layar kecil (ponsel), dan kemudian "melebar" secara adaptif untuk tablet dan desktop. Kita tidak perlu pusing menulis *media query* yang rumit untuk membuat situs kita responsif.
2. Kecepatan Pengembangan: Butuh tombol? Cukup gunakan `.btn`. Butuh *navbar*? Salin-tempel kode *navbar* dari dokumentasi. Hal ini memungkinkan *prototyping* dan pengembangan yang sangat cepat.
3. Konsistensi Visual: Semua komponen Bootstrap (tombol, *card*, *form*) dirancang untuk terlihat serasi. Hal ini memberikan tampilan yang bersih, modern, dan profesional pada aplikasi secara instan, tanpa perlu seorang desainer UI/UX khusus.
4. Kompatibilitas Lintas Peramban (Cross-Browser): Salah satu bagian tersulit dari CSS adalah memastikan tampilan situs konsisten di berbagai peramban (Chrome, Firefox, Safari, Edge). Tim Bootstrap telah melakukan pekerjaan berat ini, memastikan komponen mereka berfungsi dengan baik di mana-mana.
5. Ekosistem dan Dokumentasi: Bootstrap memiliki dokumentasi yang sangat baik dan komunitas yang besar. Hampir setiap masalah yang mungkin kita temui sudah pernah ditanyakan dan dijawab secara daring.

### 10.3 Kegiatan Praktikum

#### Langkah 1: Persiapan

1. Buka kembali folder django hasil praktikum pada Bab sebelumnya
2. Aktifkan *virtual environment* pada folder tersebut tersebut.

Terminal

```
myenv\Scripts\activate.bat
```

#### Langkah 2 : Instalasi Library

1. Tambahkan library pillow dan tinymce pada requirement.txt kemudian lakukan install

requirement.txt

```
1. Django~=5.1.2
2. Pillow~=10.4.0
3. django-tinymce~=4.1.0
```

2. Lakukan instalasi melalui pip

Terminal

```
1. pip install -r requirements.txt
```

### Langkah 3: Konfigurasi Media dan TinyMCE

1. Buka mysite/settings.py. Tambahkan 'tinymce' ke dalam daftar INSTALLED\_APPS.

```
mysite/settings.py.
2. # mysite/settings.py
3. INSTALLED_APPS = [
4.     # ...
5.     'blog',
6.     'tinymce',
7. ]
```

2. Di bagian bawah file settings.py, tambahkan konfigurasi untuk MEDIA\_URL dan MEDIA\_ROOT.

```
1. # mysite/settings.py
2. # ... (di bagian paling bawah file)
3.
4. # Konfigurasi untuk file yang diunggah pengguna
5. MEDIA_URL = '/media/'
6. MEDIA_ROOT = BASE_DIR / 'media'
```

3. Buka mysite/urls.py. Modifikasi file tersebut agar terlihat seperti pada baris kuning:

mysite/urls.py

```
1. from django.contrib import admin
2. from django.urls import path, include
3. from django.contrib.auth import views
4. from blog import views as blog_views
5. from django.conf import settings
6. from django.conf.urls.static import static
```

```

7.
8. urlpatterns = [
9.     path('admin/', admin.site.urls),
10.    path('accounts/login/', views.LoginView.as_view(), name='login'),
11.    path('accounts/logout/', blog_views.custom_logout, name='logout'),
12.    path('', include('blog.urls')),
13.]
14.
15.if settings.DEBUG:
16.    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA
    _ROOT)

```

4. Buka `blog/models.py`. Impor `HTMLField` dari `tinymce.models`. Ganti `text = models.TextField()` dengan `text = HTMLField()`. Tambahkan `ImageField` baru bernama `header_image`

`blog/models.py`

```

1. # blog/models.py
2. from django.db import models
3. from django.conf import settings
4. from django.utils import timezone
5. from tinymce.models import HTMLField # Impor ini
6.
7. class Post(models.Model):
8.     author = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.
    CASCADE)
9.     title = models.CharField(max_length=200)
10.    # Ganti TextField dengan HTMLField
11.    text = HTMLField()
12.    created_date = models.DateTimeField(default=timezone.now)
13.    published_date = models.DateTimeField(blank=True, null=True)
14.    # Tambahkan field gambar
15.    header_image = models.ImageField(upload_to='post_images/', blank=True,
    null=True)
16.
17.    # ... (sisa method publish dan __str__ tetap sama)

```

5. Jalankan migrasi. Karena kita menambahkan *field* baru, kita harus memperbarui basis data.

Terminal

```

1. python manage.py makemigrations blog
2. python manage.py migrate

```

## Langkah 4: Modifikasi Form

1. Buka `blog/forms.py`. Tambahkan `header_image` ke daftar `fields` pada `PostForm`.

`blog/forms.py`.

```
1. # blog/forms.py
2. class PostForm(forms.ModelForm):
3.     class Meta:
4.         model = Post
5.         fields = ('title', 'text', 'header_image',) # Tambahkan header_image
```

2. Buka `blog/views.py`. Tambahkan `request.FILES` ke method `post_new`

`blog/views.py`.

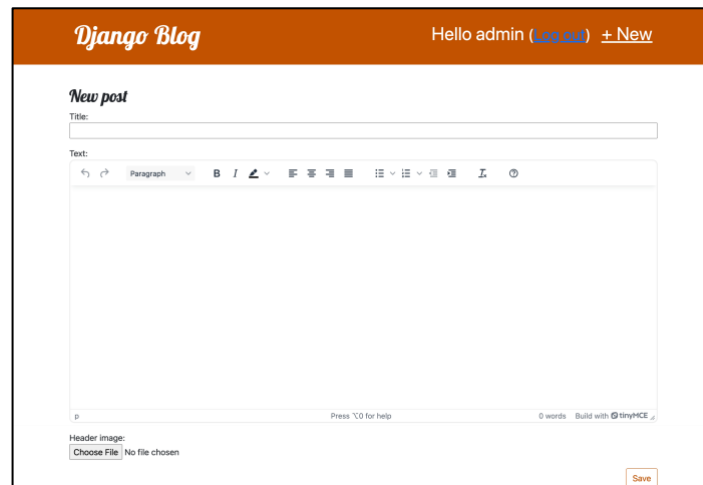
```
1. #kode sebelumnya
2. @login_required
3. def post_new(request):
4.     if request.method == "POST":
5.         form = PostForm(request.POST, request.FILES)
6.         if form.is_valid():
7.             post = form.save(commit=False)
8.             post.author = request.user
9.
10. #kode berikutnya
```

3. Buka `blog/templates/blog/post_edit.html`. Tambahkan baris 4 dan tambahkan `enctype="multipart/form-data"` pada baris 6

`blog/templates/blog/post_edit.html`

```
1. {% extends 'blog/base.html' %}
2.
3. {% block content %}
4. {{ form.media }}
5.     <h2>New post</h2>
6.     <form method="POST" class="post-form" enctype="multipart/form-data">
7.         {% csrf_token %}
8.         {{ form.as_p }}
9.         <button type="submit" class="save btn btn-secondary">Save</button>
10.     </form>
11. {% endblock %}
```

4. Jalankan server, login menggunakan akun admin. Coba lakukan edit posting atau buat postingan baru. Sekarang tampilan input postingan akan memiliki fitur formating text dan muncul tempat untuk unggah header image



#### Langkah 4: Implementasi Bootstrap pada blog

5. Buka `blog/templates/blog/base.html`. Tambahkan kode untuk footer sebelum tag `</body>`

`blog/templates/blog/base.html`

```
1. #kode sebelumnya
2. <footer class="bg-dark text-white text-center p-4 mt-5">
3.     <div class="container">
4.         <p>&copy; 2024 - Pendidikan Teknik Informatika, UMS</p>
5.     </div>
6. </footer>
7. </body>
8. </html>
```

6. Buka `blog/templates/blog/post_list.html`. Ganti seluruh isinya dengan kode baru ini yang menggunakan komponen Bootstrap.

`blog/templates/blog/post_list.html`

```
1. {% extends 'blog/base.html' %}
2.
3. {% block content %}
4. <!-- Hero Section -->
5. <div class="hero-section text-center p-5 mb-4">
6.     <div class="container">
7.         <h1 class="display-4">Selamat Datang di Blog PTI</h1>
```

```

8.         <p class="lead">Tempat berbagi informasi seputar teknologi dan pen
        didikan.</p>
9.     </div>
10.</div>
11.
12.<!-- Kontainer untuk Card Postingan -->
13.<div class="container">
14.    <div class="row">
15.        <!-- Loop Postingan -->
16.        {% for post in posts %}
17.            <div class="col-md-6 col-lg-4 mb-4">
18.                <div class="card h-100 shadow-sm">
19.                    <!-- Tampilkan gambar jika ada -->
20.                    {% if post.header_image %}
21.                        
22.                    {% else %}
23.                        <!-- Placeholder jika tidak ada gambar -->
24.                        
25.                    {% endif %}
26.
27.                    <div class="card-body d-flex flex-column">
28.                        <h5 class="card-title">{{ post.title }}</h5>
29.                        <p class="card-text">
30.                            <!-- Gunakan filter |safe dan |truncatewords -->
31.                            {{ post.text|safe|truncatewords:20 }}
32.                        </p>
33.                        <a href="{% url 'post_detail' pk=post.pk %}" class="bt
                n btn-primary mt-auto">Baca Selengkapnya</a>
34.                    </div>
35.                    <div class="card-footer text-muted">
36.                        Dipublikasi pada {{ post.published_date }}
37.                    </div>
38.                </div>
39.            </div>
40.        {% empty %}
41.            <p>Belum ada postingan.</p>
42.        {% endfor %}
43.    </div>
44.</div>
45.{% endblock %}
46.

```

7. Buka `blog/templates/blog/post_detail.html`. Ganti juga isinya agar sesuai dengan *styling* Bootstrap.

`blog/templates/blog/post_detail.html`.

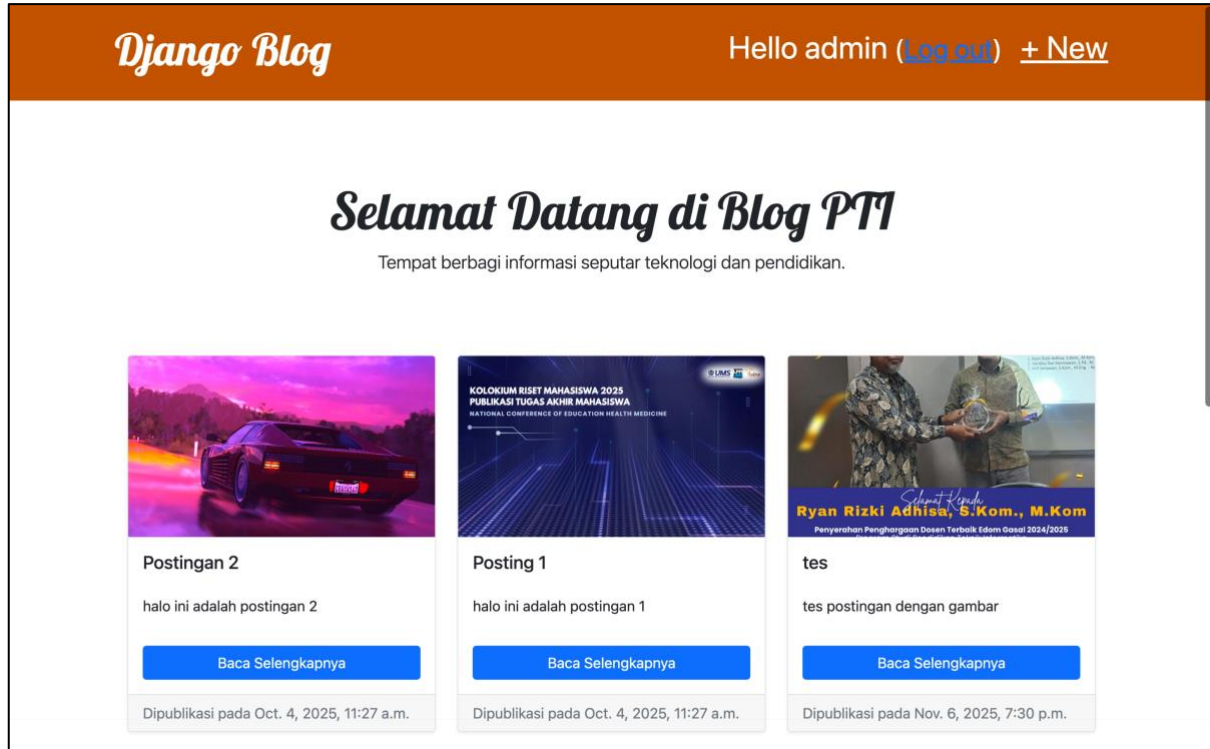
```

1. {% extends 'blog/base.html' %}
2. {% block content %}
3. <div class="container mt-5">
4.     <div class="row">
5.         <div class="col-lg-10 mx-auto">
6.             <!-- Judul Postingan -->
7.             <h1 class="display-5">{{ post.title }}</h1>
8.             <p class="text-muted">
9.                 Oleh: {{ post.author }} | Dipublikasi: {{ post.published_date }}
10.                {% if user.is_authenticated %}
11.                <a
12.                    class="btn btn-outline-secondary btn-sm"
13.                    href="{% url 'post_edit' pk=post.pk %}"
14.                >Edit</a>
15.                {% endif %}
16.            </p>
17.
18.            <!-- Gambar Header -->
19.            {% if post.header_image %}
20.            
25.            {% endif %}
26.
27.            <!-- Konten Postingan (Rich Text) -->
28.            <div class="post-content">{{ post.text|safe }}</div>
29.            <hr class="my-5" />
30.            <a
31.                class="btn btn-secondary"
32.                role="button"
33.                href="{% url 'add_comment_to_post' pk=post.pk %}"
34.            >Add comment</a>
35.        >
36.
37.        <hr class="my-5" />
38.        {% for comment in post.comments.all %}
39.        <div class="card bg-light mb-3">
40.            <div class="card-body">
41.                {{ comment.created_date }}
42.                <strong>{{ comment.author }}</strong>
43.                <p>{{ comment.text|linebreaks }}</p>
44.            </div>
45.        </div>
46.        {% empty %}
47.        <p>No comments here yet :(</p>
48.        {% endfor %}
49.    </div>
50. </div>
51.</div>

```

```
52.{% endblock %}
```

8. Jalankan server dan tampilan website akan menjadi seperti berikut



## 10.4 Rangkuman

- Kita telah memisahkan antara file statis (`STATIC`) dan file media (`MEDIA`). File media adalah file yang diunggah pengguna dan memerlukan konfigurasi `MEDIA_ROOT` dan `MEDIA_URL`.
- *Library* Pillow diperlukan oleh Django untuk memproses *upload* gambar (`ImageField`).
- Formulir HTML yang mengunggah file wajib menyertakan atribut `enctype="multipart/form-data"`.
- `django-tinymce` memungkinkan kita mengganti `TextField` standar dengan *rich text editor* (WYSIWYG) dengan mudah, biasanya menggunakan `HTMLField` dari `tinymce.models`.
- Konten HTML dari *rich text editor* harus dirender di *template* menggunakan *filter* `|safe` agar tag HTML dapat diterjemahkan oleh peramban.
- Bootstrap adalah *framework* CSS yang mempercepat pengembangan *frontend*. Kita dapat menggunakan komponen-komponennya seperti `navbar`, `container`, `row`, `col`, dan `card` untuk tampilan yang modern dan responsif.

## 10.5 Tugas

Kerjakan tugas-tugas di bawah ini untuk memperkuat pemahaman dan keterampilan Anda.

1. Jelaskan perbedaan fundamental antara `STATIC_ROOT` dan `MEDIA_ROOT` dalam konfigurasi Django.
2. Apa risiko keamanan yang mungkin timbul saat menggunakan *filter* `|safe` di *template* Django?